August 25, 2026 at 09:54

1. Introduction. This program counts **open** knight's tours (open Hamiltonian paths) of the $m \times n$ knight graph by a **dual-frontier** transfer that is aggregated with **bucket sorting** rather than a trie, so it parallelizes; and it extracts the **periodic** transfer once the frontier stabilizes, so that far columns are reached by a cheap sparse matrix-vector product (SpMV) instead of re-running the generator.

The method is Knuth's (as in `DYNAHAM/dynahamp`), reimplemented from scratch. Work in the augmented graph $G^+ = G + K_1$: a new **apex** vertex joins every cell, and an open tour of G is a Hamiltonian cycle of G^+ through the apex. Place the cells column by column. After s cells are placed, the **frontier** is the set of not-yet-placed vertices adjacent to a placed one, plus the apex. Each frontier vertex is **bare** (degree 0), **outer** (degree 1, a subpath endpoint), or **inner** (degree 2). The state is that pattern together with the pairing of the outer endpoints. Because it is coded by **relative** positions, not absolute vertex numbers, the code is translation-invariant: the sorted state set repeats with some small period once we are in the bulk, which is exactly what makes the transfer periodic and the SpMV possible.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>
#include <sys/resource.h>
#include <unistd.h>
#include <pthread.h>
```

⟨ Types 2 ⟩
⟨ Globals 3 ⟩
⟨ Subroutines 4 ⟩
⟨ Main 29 ⟩

2. Board and frontier. Cell (r,c) is vertex $v = cm + r$; there are $V = mn$ cells and the apex is V. *frontier_before*(s) lists (sorted) the frontier just before cell s: the apex, s itself (an "extended" frontier, always present), and each future cell $> s$ with an already-placed neighbour.

⟨ Types 2 ⟩ ≡
  typedef unsigned long long u64;

See also section 10.

This code is used in section 1.

3. **#define** `MAXF` 512
#define `MAXLEV` 40

⟨ Globals 3 ⟩ ≡
  int m, n, V;
  int `NB`$[64 * 64][8]$, `ND`$[64 * 64]$;
  static const int `KR`$[8] = \{-2, -2, -1, -1, 1, 1, 2, 2\}$;
  static const int `KC`$[8] = \{-1, 1, -2, 2, -2, 2, -1, 1\}$;
  int *fr*[`MAXF`], *ifrb*$[64 * 64 + 2]$;

See also sections 5, 8, 11, and 13.

This code is used in section 1.

4. All large allocations go through checked wrappers: on failure they print and *exit* gracefully, so a run can never provoke the kernel's OOM killer.

⟨ Subroutines 4 ⟩ ≡

```
void *xmalloc(size_t n)
{
  void *p = malloc(n);
  if (¬p ∧ n) {
    fprintf(stderr, "OOM:␣malloc␣%zu␣bytes␣failed\n", n);
    exit(3);
  }
  return p;
}
void *xrealloc(void *q, size_t n)
{
  void *p = realloc(q, n);
  if (¬p ∧ n) {
    fprintf(stderr, "OOM:␣realloc␣%zu␣bytes␣failed\n", n);
    exit(3);
  }
  return p;
}
void *xcalloc(size_t a, size_t b)
{
  void *p = calloc(a, b);
  if (¬p ∧ a ∧ b) {
    fprintf(stderr, "OOM:␣calloc␣%zu*%zu␣failed\n", a, b);
    exit(3);
  }
  return p;
}
```

See also sections 6, 7, 9, 12, 14, 15, 19, 21, 23, and 26.
This code is used in section 1.

5. To bound *physical* memory – the resident set size, which is what actually triggers the kernel OOM killer – a detached watcher thread samples the RSS and *_exit*(3)s gracefully if it exceeds the cap. We deliberately do **not** cap the virtual address space with RLIMIT_AS: glibc arenas, per-thread stacks, and *qsort*'s scratch *mmap* reserve far more virtual than resident memory, so an RLIMIT_AS cap makes those (non-*xmalloc*) allocations fail with a hard crash instead of a clean exit. RSS is the quantity that matters for OOM, and the largest single allocation here (an edge table, a few GB) fits comfortably inside the headroom between the 85%-RAM cap and total RAM, so the 0.5 s sampling never lets resident memory overshoot the machine.

⟨ Globals 3 ⟩ +≡

```
long mem_cap_bytes;
```

6. ⟨ Subroutines 4 ⟩ +≡

```
static long rss_bytes(void)
{
  FILE *f = fopen("/proc/self/statm", "r");
  if (¬f) return 0;
  long sz = 0, res = 0;
  if (fscanf(f, "%ld␣%ld", &sz, &res) ≠ 2) res = 0;
  fclose(f);
  return res * (long) sysconf(_SC_PAGE_SIZE);
}
static void *mem_watcher(void *arg)
{
  (void) arg;
  for ( ; ; ) {
    long r = rss_bytes();
    if (mem_cap_bytes > 0 ∧ r > mem_cap_bytes) {
      fprintf(stderr,
          "\nMEMORY␣CAP:␣RSS␣%.1f␣GB␣exceeds␣cap␣%.1f␣GB␣--␣exiting""␣gracefully␣(raise␣\
          MEMCAP_GB␣to␣allow␣more)\n", r/1073741824.0, mem_cap_bytes/1073741824.0);
      fflush(stderr);
      _exit(3);
    }
    usleep(500000);
  }
  return 0;
}
void start_mem_watcher(void)
{
  long ram = sysconf(_SC_PHYS_PAGES) * (long) sysconf(_SC_PAGE_SIZE);
  mem_cap_bytes = (long)(ram * 0.85);
  char *e = getenv("MEMCAP_GB");
  if (e) mem_cap_bytes = atol(e) * (1L ≪ 30);
  pthread_t th;
  pthread_create(&th, 0, mem_watcher, 0);
  pthread_detach(th);
  fprintf(stderr, "self␣memory␣cap:␣%.0f␣GB␣RSS␣(set␣MEMCAP_GB␣to␣override)\n",
      mem_cap_bytes/1073741824.0);
}
```

7. ⟨ Subroutines 4 ⟩ +≡

```
void build_board(void)
{
  int r, c, k;
  V = m * n;
  for (c = 0; c < n; c++)
    for (r = 0; r < m; r++) {
      int v = c * m + r;
      ND[v] = 0;
      for (k = 0; k < 8; k++) {
        int rr = r + KR[k], cc = c + KC[k];
        if (rr >= 0 && rr < m && cc >= 0 && cc < n) NB[v][ND[v]++] = cc * m + rr;
      }
    }
}
int frontier_before(int s)
{
  int q = 0, v, u, k;
  static char inF[64 * 64 + 2];
  for (v = 0; v <= V; v++) inF[v] = 0;
  inF[V] = 1;
  if (s < V) inF[s] = 1;
  for (v = s + 1; v < V; v++)
    for (k = 0; k < ND[v]; k++) {
      u = NB[v][k];
      if (u < s) {
        inF[v] = 1;
        break;
      }
    }
  for (v = 0; v <= V; v++)
    if (inF[v]) {
      fr[q] = v;
      ifrb[v] = q;
      q++;
    }
  return q;
}
```

8. Mate table, derived edges, key, completion. The state is *mate*[ ] over frontier positions: -2 bare, -1 inner, else the partner position (outer). *add_derived*(i, j) joins the subpaths ending at i, j; if they are the two ends of one subpath a cycle forms (flagged in *cycle*, both ends set inner). The key is one byte per position; equal (also column-shifted) patterns give equal keys. A cycle through the apex is a valid open m'-tour iff the covered (inner) cells are the contiguous prefix $\{0..m'-1\}$ and the rest of the board frontier is bare; *completion_mp* returns that m'.

⟨ Globals 3 ⟩ +≡
  int *mate*[MAXF];
  int *cycle*;
#**pragma** *omp threadprivate* (*mate*, *cycle*)

9. ⟨ Subroutines 4 ⟩ +≡

```
int add_derived(int i, int j)
{
  cycle = 0;
  if (mate[i] ≡ −1 ∨ mate[j] ≡ −1) return 0;
  if (mate[i] ≡ −2) {
    if (mate[j] ≡ −2) {
      if (i ≡ j) {
        cycle = 1;
        return 0;
      }
      mate[i] = j;
      mate[j] = i;
      return 1;
    }
    mate[i] = mate[j];
    mate[mate[j]] = i;
    mate[j] = −1;
    return 1;
  }
  else if (mate[j] ≡ −2) {
    mate[j] = mate[i];
    mate[mate[i]] = j;
    mate[i] = −1;
    return 1;
  }
  else if (mate[i] ≠ j) {
    mate[mate[i]] = mate[j];
    mate[mate[j]] = mate[i];
    mate[i] = mate[j] = −1;
    return 1;
  }
  mate[i] = mate[j] = −1;
  cycle = 1;
  return 0;
}
int keyof(int q, unsigned char *key)
{
  int i;
  for (i = 0; i < q; i++) key[i] = mate[i] ≡ −2 ? 0 : mate[i] ≡ −1 ? 255 : (unsigned char)(1 + mate[i]);
  return q;
}
int completion_mp(int q, int apexpos, int *frn, int s)
{
  int k, mp = s + 1;
  if (mate[apexpos] ≠ −1) return 0;
  k = 0;
  while (k < q ∧ frn[k] ≠ V ∧ mate[k] ≡ −1 ∧ frn[k] ≡ mp) {
    mp++;
    k++;
  }
```

```
  for ( ; k < q; k++) {
    if (frn[k] ≡ V) continue;
    if (mate[k] ≠ −2) return 0;
  }
  return mp;
}
```

10. Bucket aggregation. Each step emits successor records; we sort by key and merge equal keys. *src* carries the emitting state's index, used only while recording the edge tables.

```
⟨ Types 2 ⟩ +≡
  typedef struct {
    long off;
    int len;
    u64 w;
    long src;
  } Rec;
```

11. ⟨ Globals 3 ⟩ +≡

```
  unsigned char *kp;
  long kcap, kuse;
  Rec *rc;
  long nr, rcap;
  unsigned char *cb;
#define NTMAX 128
  unsigned char *tkp[NTMAX];
  long tkuse[NTMAX], tkcap[NTMAX];
  Rec *trc[NTMAX];
  long tnr[NTMAX], trcap[NTMAX];
  int rcmp(const void *A, const void *B)
  {
    const Rec *a = A, *b = B;
    int l = a->len < b->len ? a->len : b->len;
    int d = memcmp(cb + a->off, cb + b->off, l);
    return d ? d : a->len - b->len;
  }
```

12. ⟨ Subroutines 4 ⟩ +≡

```
void emit(unsigned char *key, int len, u64 w, long src)
{
  int t = omp_get_thread_num();
  if (tkuse[t] + len > tkcap[t]) {
    tkcap[t] = tkcap[t] * 2 + len + 65536;
    tkp[t] = xrealloc(tkp[t], tkcap[t]);
  }
  if (tnr[t] >= trcap[t]) {
    trcap[t] = trcap[t] * 2 + 65536;
    trc[t] = xrealloc(trc[t], trcap[t] * sizeof(Rec));
  }
  memcpy(tkp[t] + tkuse[t], key, len);
  trc[t][tnr[t]].off = tkuse[t];
  trc[t][tnr[t]].len = len;
  trc[t][tnr[t]].w = w;
  trc[t][tnr[t]].src = src;
  tkuse[t] += len;
  tnr[t]++;
}
void merge_pools(void)
{
  int t;
  long r;
  nr = 0;
  kuse = 0;
  for (t = 0; t < omp_get_max_threads(); t++) {
    for (r = 0; r < tnr[t]; r++) {
      Rec *R = &trc[t][r];
      if (kuse + R->len > kcap) {
        kcap = kcap * 2 + R->len + 65536;
        kp = xrealloc(kp, kcap);
      }
      if (nr >= rcap) {
        rcap = rcap * 2 + 65536;
        rc = xrealloc(rc, rcap * sizeof(Rec));
      }
      memcpy(kp + kuse, tkp[t] + R->off, R->len);
      rc[nr].off = kuse;
      rc[nr].len = R->len;
      rc[nr].w = R->w;
      rc[nr].src = R->src;
      kuse += R->len;
      nr++;
    }
    tnr[t] = 0;
    tkuse[t] = 0;
  }
}
int rcmp_r(const void *A, const void *B, void *arg)
{
```

```
    unsigned char *base = arg;
    const Rec *a = A, *b = B;
    int l = a→len < b→len ? a→len : b→len;
    int d = memcmp(base + a→off, base + b→off, l);
    return d ? d : a→len − b→len;
  }   /* sort each thread's records in parallel, then merge+reduce across them in one pass (keeping global
       sort order so periodic ids stay consistent), capturing the integer edge table when recording. */
  long in_ncur_g;   /* Merge the per-thread sorted lists with an NT-way heap directly into the reduced
       output – memory-efficient (no full sorted copy is materialized), so it stays within RAM for large
       m. It is partly serial (the heap pops); a splitter-based PARALLEL merge (partition the output by
       sampled splitters, then merge each part independently) is the next optimization for m¿=7 speed, but
       the bucket-copy variant it replaces used  2x the memory and could OOM. */
  void sort_reduce(int s, int rec) { int NT = omp_get_max_threads( ), t;
      long tot = 0;
#pragma omp parallel for schedule (dynamic, 1)
      for (t = 0; t < NT; t++)
        if (tnr[t]) qsort_r(trc[t], tnr[t], sizeof(Rec), rcmp_r, tkp[t]);
      for (t = 0; t < NT; t++) tot += tnr[t];
      long tbytes = 0;
      for (t = 0; t < NT; t++) tbytes += tkuse[t];
      curoff = xrealloc(curoff, (tot + 1) * sizeof(long));
      curkl = xrealloc(curkl, (tot + 1) * sizeof(int));
      curw = xrealloc(curw, (tot + 1) * sizeof(u64));
      curkp = xrealloc(curkp, tbytes + 1);
      static long idx[NTMAX];
      for (t = 0; t < NT; t++) idx[t] = 0;
      Edge * eb = 0;
      long enb = 0;
      if (rec) eb = xmalloc((tot + 1) * sizeof (Edge));
      long k = 0, use = 0;
      static int heap[NTMAX];
      int hn = 0;
#define HKEY(t) (tkp[t] + trc[t][idx[t]].off)
#define HLEN(t) (trc[t][idx[t]].len)
#define HLESS(a, b) (
      {
        int la_ = HLEN(a), lb_ = HLEN(b), l_ = la_ < lb_ ? la_ : lb_;
        int d_ = memcmp(HKEY(a), HKEY(b), l_);
        d_ < 0 ∨ (d_ ≡ 0 ∧ la_ < lb_);
      }
      )
      for (t = 0; t < NT; t++)
        if (idx[t] < tnr[t]) {
          int c = hn++;
          heap[c] = t;
          while (c > 0) {
            int pp = (c − 1)/2;
            if (HLESS(heap[c], heap[pp])) {
              int x = heap[c];
```

```
          heap[c] = heap[pp];
          heap[pp] = x;
          c = pp;
        }
        else break;
      }
    }
  int curlen = −1;
  long curpos = 0;
  u64 sw = 0;
  while (hn > 0) {
    int mt = heap[0];
    Rec *R = &trc[mt][idx[mt]];
    unsigned char *rkey = tkp[mt] + R→off;
    int rlen = R→len;
    int same = (curlen ≡ rlen ∧ memcmp(curkp + curpos, rkey, rlen) ≡ 0);
    if (¬same) {
      if (curlen ≥ 0) {
        curw[k] = sw;
        k++;
      }
      curpos = use;
      memcpy(curkp + use, rkey, rlen);
      curoff[k] = use;
      curkl[k] = rlen;
      use += rlen;
      curlen = rlen;
      sw = 0;
    }
    sw = red(sw + R→w);
    if (rec) {
      eb[enb].src = (int) R→src;
      eb[enb].dst = (int) k;
      eb[enb].c = 1;
      enb++;
    }
    idx[mt]++;
    if (idx[mt] ≥ tnr[mt]) heap[0] = heap[−−hn];
    {
      int c = 0;
      while (1) {
        int l = 2 * c + 1, r = 2 * c + 2, sm = c;
        if (l < hn ∧ HLESS(heap[l], heap[sm])) sm = l;
        if (r < hn ∧ HLESS(heap[r], heap[sm])) sm = r;
        if (sm ≡ c) break;
        int x = heap[c];
        heap[c] = heap[sm];
        heap[sm] = x;
        c = sm;
      }
```

```
    }
  }
  if (curlen ≥ 0) {
    curw[k] = sw;
    k++;
  }
  ncur = k;
  for (t = 0; t < NT; t++) {
    tnr[t] = 0;
    tkuse[t] = 0;
  }
  if (rec) {
    long o = 0, p;
    qsort(eb, enb, sizeof (Edge), ecmp);
    for (p = 0; p < enb; ) {
      long q2 = p + 1;
      u64 cc = 1;
      while (q2 < enb ∧ eb[q2].src ≡ eb[p].src ∧ eb[q2].dst ≡ eb[p].dst) {
        cc++;
        q2++;
      }
      eb[o] = eb[p];
      eb[o].c = cc;
      o++;
      p = q2;
    }
    edges[reclev] = xrealloc(eb, (o + 1) ∗ sizeof (Edge));
    nedge[reclev] = o;
    nstate[reclev] = in_ncur_g;
    nstate[reclev + 1] = ncur;
    comps[reclev] = xmalloc((reccomp_n + 1) ∗ sizeof (Comp));
    memcpy(comps[reclev], reccomp_buf, reccomp_n ∗ sizeof (Comp));
    ncomp[reclev] = reccomp_n;
    reclev++;
  }
  }
```

13. The transfer step. The bucket holds states as (key,weight) over the previous frontier. To place cell s: build the base mate table *bmate* over the new frontier (survivors carried, newcomers bare) with s in a temporary slot, then bring s to degree 2 by adding its $2 - \deg(s)$ edges to future neighbours or the apex. When *rec* is set we also record the integer edge table (src index → dst index) and the completion contributions for this level.

⟨ Globals 3 ⟩ +≡

```
unsigned char *curkp;
long *curoff;
int *curkl;
u64 *curw;
long ncur;
u64 cnt[1 ≪ 16];
u64 MODP = 0;    /* 0 = exact u64; set to a prime for one CRT residue */
static inline u64 red(u64 x)
{
  return MODP ? x % MODP : x;
}
int qnew, posS, apexnew, STEMP;
int bmate[MAXF], o2n[MAXF];
#pragma omp threadprivate (bmate, STEMP)
int recording, reclev;
typedef struct {
  int src, dst;
  u64 c;
} Edge;
typedef struct {
  int src, delta;
  u64 mult;
} Comp;
Edge *edges[MAXLEV];
long nedge[MAXLEV];
Comp *comps[MAXLEV];
long ncomp[MAXLEV];
long nstate[MAXLEV + 1];
int Plevs;
int period_g, c0_g, recend_col_g;    /* saved for dumping the extracted tables */
int direct_valid_col_g;    /* direct cnt is exact for columns ≤ this */
int stop_after_record = 0;
int direct_cov_g = 0;
static u64 colfp_g[4096];
const char *ckpt_path = 0;
int ckpt_every = 4;    /* checkpoint every K columns */
u64 *seedv;
long seedn;
Comp *reccomp_buf;
long reccomp_n, reccomp_cap;
int ecmp(const void *A, const void *B)
{
  const Edge *a = A, *b = B;
  if (a→dst ≠ b→dst) return a→dst − b→dst;
  return a→src − b→src;
```

```
}
```

14. ⟨ Subroutines 4 ⟩ +≡

```
void build_bmate(int *omate, int qold)
{
  int i;
  for (i = 0; i ≤ qnew; i++) bmate[i] = −2;
  STEMP = qnew;
  for (i = 0; i < qold; i++) {
    int dst = (i ≡ posS) ? STEMP : o2n[i];
    if (dst < 0) continue;
    if (omate[i] ≡ −1) bmate[dst] = −1;
    else if (omate[i] ≥ 0) {
      int op = omate[i];
      int pdst = (op ≡ posS) ? STEMP : o2n[op];
      bmate[dst] = pdst ≥ 0 ? pdst : −2;
    }
  }
}
```

15. ⟨ Subroutines 4 ⟩ +≡

```
void run_step(int s, int rec)
{
  int i;
  long si;
  long in_ncur = ncur;
  int qold = frontier_before(s);
  posS = ifrb[s];
  static int frold[MAXF];
  for (i = 0; i < qold; i++) frold[i] = fr[i];
  qnew = frontier_before(s + 1);
  static int ifrnew[64 * 64 + 2], frnew[MAXF];
  for (i = 0; i < qnew; i++) frnew[i] = fr[i];
  for (i = 0; i ≤ V; i++) ifrnew[i] = −1;
  for (i = 0; i < qnew; i++) ifrnew[frnew[i]] = i;
  apexnew = ifrnew[V];
  int nbr[16], rr = 0;
  nbr[rr++] = apexnew;
  for (i = 0; i < ND[s]; i++) {
    int w = NB[s][i];
    if (w > s ∧ ifrnew[w] ≥ 0) nbr[rr++] = ifrnew[w];
  }
  for (i = 0; i < qold; i++) o2n[i] = (frold[i] ≡ s) ? −1 : ifrnew[frold[i]];
  in_ncur_g = ncur;
  reccomp_n = 0;
  ⟨ Expand every state at cell s 16 ⟩;
  ⟨ Sort, reduce, and (if recording) capture the edge table 18 ⟩;
}
```

16. Each state expands independently, so the loop runs in parallel (except while recording, when it stays serial to keep the completion log ordered). The transition scratch (*mate*, *bmate*, *cycle*, `STEMP`) is *threadprivate*; each thread emits into its own pool; *cnt* is updated in a critical section.

```
⟨ Expand every state at cell s 16 ⟩ ≡
#pragma omp parallel for schedule (dynamic, 32)
  for (si = 0; si < ncur; si++) {
    int i, a, b, nl, deg, need, mp;
    unsigned char *ok = curkp + curoff[si];
    int okl = curkl[si];
    u64 w = curw[si];
    int omate[MAXF];
    unsigned char nk[MAXF];
    for (i = 0; i < okl; i++) {
      int cc = ok[i];
      omate[i] = cc ≡ 0 ? −2 : cc ≡ 255 ? −1 : cc − 1;
    }
    deg = omate[posS] ≡ −2 ? 0 : omate[posS] ≡ −1 ? 2 : 1;
    need = 2 − deg;
    if (need ≡ 0) {
      build_bmate(omate, qold);
      for (i = 0; i < qnew; i++) mate[i] = bmate[i];
      nl = keyof(qnew, nk);
      emit(nk, nl, w, si);
    }
    else if (need ≡ 1) {
      for (a = 0; a < rr; a++) {
        build_bmate(omate, qold);
        for (i = 0; i ≤ qnew; i++) mate[i] = bmate[i];
        if (add_derived(STEMP, nbr[a])) {
          nl = keyof(qnew, nk);
          emit(nk, nl, w, si);
        }
        else if (cycle) {
          mp = completion_mp(qnew, apexnew, frnew, s);
          if (mp) ⟨ Credit completion 17 ⟩;
        }
      }
    }
    else {
      for (a = 0; a < rr; a++)
        for (b = a + 1; b < rr; b++) {
          build_bmate(omate, qold);
          for (i = 0; i ≤ qnew; i++) mate[i] = bmate[i];
          if (¬add_derived(STEMP, nbr[a])) {
            if (cycle) {
              mp = completion_mp(qnew, apexnew, frnew, s);
              if (mp) ⟨ Credit completion 17 ⟩;
            }
            continue;
          }
          if (add_derived(STEMP, nbr[b])) {
```

```
          nl = keyof(qnew, nk);
          emit(nk, nl, w, si);
        }
        else if (cycle) {
          mp = completion_mp(qnew, apexnew, frnew, s);
          if (mp) ⟨Credit completion 17⟩;
        }
      }
    }
  }
```

This code is used in section 15.

17. ⟨Credit completion 17⟩ ≡

```
  {
#pragma omp critical
    {
      cnt[mp] = red(cnt[mp] + w);
      if (rec) {
        if (reccomp_n ≥ reccomp_cap) {
          reccomp_cap = reccomp_cap * 2 + (1 ≪ 20);
          reccomp_buf = xrealloc(reccomp_buf, reccomp_cap * sizeof(Comp));
        }
        reccomp_buf[reccomp_n].src = si;
        reccomp_buf[reccomp_n].delta = mp − (s + 1);
        reccomp_buf[reccomp_n].mult = 1;
        reccomp_n++;
      }
    }
  }
```

This code is used in section 16.

18. The sort and reduce run across the per-thread pools in parallel (see *sort_reduce*); *in_ncur_g* records the input level size for the edge table.

⟨Sort, reduce, and (if recording) capture the edge table 18⟩ ≡

```
  sort_reduce(s, rec);
```

This code is used in section 15.

19. Checkpointing the build. The build (the sweep that discovers the periodic transfer) is the long part; we checkpoint it at every column boundary before recording begins, dumping the current bucket, the running counts, and the fingerprints. A build that dies resumes from the last boundary; the recording columns themselves are few, so if a crash lands inside them we simply restart from the last pre-recording checkpoint.

⟨ Subroutines 4 ⟩ +≡

```
void save_ckpt(int nexts)
{
  if (¬ckpt_path) return;
  FILE *f = fopen(ckpt_path, "wb");
  fwrite(&m, sizeof(int), 1, f);
  fwrite(&nexts, sizeof(int), 1, f);
  fwrite(cnt, sizeof(u64), 1 ≪ 16, f);
  fwrite(colfp_g, sizeof(u64), 4096, f);
  fwrite(&ncur, sizeof(long), 1, f);
  fwrite(&kuse, sizeof(long), 1, f);
  fwrite(curoff, sizeof(long), ncur, f);
  fwrite(curkl, sizeof(int), ncur, f);
  fwrite(curw, sizeof(u64), ncur, f);
  {
    long kb = 0, i;
    for (i = 0; i < ncur; i++) kb += curkl[i];
    fwrite(&kb, sizeof(long), 1, f);
    fwrite(curkp, 1, kb, f);
  }
  fclose(f);
}
int load_ckpt(const char *path)
{
  FILE *f = fopen(path, "rb");
  if (¬f) return −1;
  int nexts, mm;
  if (fread(&mm, sizeof(int), 1, f) ≠ 1) return −1;
  m = mm;
  fread(&nexts, sizeof(int), 1, f);
  fread(cnt, sizeof(u64), 1 ≪ 16, f);
  fread(colfp_g, sizeof(u64), 4096, f);
  fread(&ncur, sizeof(long), 1, f);
  fread(&kuse, sizeof(long), 1, f);
  curoff = xrealloc(curoff, (ncur + 1) * sizeof(long));
  curkl = xrealloc(curkl, (ncur + 1) * sizeof(int));
  curw = xrealloc(curw, (ncur + 1) * sizeof(u64));
  fread(curoff, sizeof(long), ncur, f);
  fread(curkl, sizeof(int), ncur, f);
  fread(curw, sizeof(u64), ncur, f);
  {
    long kb;
    fread(&kb, sizeof(long), 1, f);
    curkp = xrealloc(curkp, kb + 1);
    fread(curkp, 1, kb, f);
```

```
    }
    fclose(f);
    fprintf(stderr, "resumed␣build␣from␣%s␣at␣cell␣%d␣(col␣%d)\n", path, nexts, nexts/mm);
    return nexts;
}
```

20.

21. Dumping and reloading the extracted tables. The expensive part is the build that discovers and records the periodic transfer. Once recorded, the tables (edge tables, completions, seed vector, level sizes, period, and the directly-counted columns up to *recend_col_g*) are small and self-contained, so we dump them to disk. A later run reloads them and does only the cheap SpMV — and a build that dies can be re-run without touching the SpMV.

⟨ Subroutines 4 ⟩ +≡

```
void dump_tables(const char *path)
{
  FILE *f = fopen(path, "wb");
  int L;
  int hdr[6] = {m, period_g, c0_g, Plevs, recend_col_g, direct_valid_col_g};
  fwrite(hdr, sizeof(int), 6, f);
  fwrite(&seedn, sizeof(long), 1, f);
  fwrite(seedv, sizeof(u64), seedn, f);
  fwrite(nstate, sizeof(long), Plevs + 1, f);
  for (L = 0; L < Plevs; L++) {
    fwrite(&nedge[L], sizeof(long), 1, f);
    fwrite(edges[L], sizeof(Edge), nedge[L], f);
    fwrite(&ncomp[L], sizeof(long), 1, f);
    fwrite(comps[L], sizeof(Comp), ncomp[L], f);
  }
  int nc = (direct_valid_col_g + 1) * m;
  fwrite(&nc, sizeof(int), 1, f);
  fwrite(cnt, sizeof(u64), nc, f);
  fclose(f);
  fprintf(stderr, "dumped␣tables␣to␣%s␣(period␣%d,␣c0␣%d,␣%d␣levels)\n", path, period_g, c0_g,
      Plevs);
}
int load_tables(const char *path)
{
  FILE *f = fopen(path, "rb");
  if (¬f) return 0;
  int L, hdr[6];
  if (fread(hdr, sizeof(int), 6, f) ≠ 6) return 0;
  m = hdr[0];
  period_g = hdr[1];
  c0_g = hdr[2];
  Plevs = hdr[3];
  recend_col_g = hdr[4];
  direct_valid_col_g = hdr[5];
  fread(&seedn, sizeof(long), 1, f);
  seedv = xmalloc(seedn * sizeof(u64));
  fread(seedv, sizeof(u64), seedn, f);
  fread(nstate, sizeof(long), Plevs + 1, f);
  for (L = 0; L < Plevs; L++) {
    fread(&nedge[L], sizeof(long), 1, f);
    edges[L] = xmalloc((nedge[L] + 1) * sizeof(Edge));
    fread(edges[L], sizeof(Edge), nedge[L], f);
    fread(&ncomp[L], sizeof(long), 1, f);
    comps[L] = xmalloc((ncomp[L] + 1) * sizeof(Comp));
```

```
      fread(comps[L], sizeof(Comp), ncomp[L], f);
    }
    int nc;
    fread(&nc, sizeof(int), 1, f);
    memset(cnt, 0, sizeof (cnt));
    fread(cnt, sizeof(u64), nc, f);
    fclose(f);
    return 1;
  }
```

22.

23. Driver: build, extract the period, then SpMV. Sweep an $m \times W_b$ strip; when the boundary key-set (fingerprinted) repeats with period $p \in \{1,2\}$ at column c_0, record the next p columns' edge tables and capture the seed vector. Then iterate the SpMV to reach column N: at each recorded level apply the edge table to the state vector and add the level's completions to *cnt2*. The two agree on every column the SpMV fully covers ($c \ge c_0 + 3$), and the SpMV alone yields the columns past W_b.

⟨ Subroutines 4 ⟩ +≡

```
int resume_from = 0;
void seed_bucket(void)
{
  int i;
  int q = frontier_before(0);
  for (i = 0; i < q; i++) mate[i] = −2;
  unsigned char key[MAXF];
  int kl = keyof(q, key);
  curkp = xmalloc(1 ≪ 20);
  curoff = xmalloc(sizeof(long));
  curkl = xmalloc(sizeof(int));
  curw = xmalloc(sizeof(u64));
  memcpy(curkp, key, kl);
  curoff[0] = 0;
  curkl[0] = kl;
  curw[0] = 1;
  ncur = 1;
}
u64 cnt2[1 ≪ 16];
int run_periodic(int mm, int Wb, int Next)
{
  int i, s, c;
  m = mm;
  n = Wb;
  build_board( );
  memset(cnt2, 0, sizeof (cnt2));
  recording = 0;
  reclev = 0;
  c0_g = −1;
  recend_col_g = −1;
  direct_valid_col_g = −1;
  Plevs = 0;
  seedn = 0;
  if (resume_from ≤ 0) {
    memset(cnt, 0, sizeof (cnt));
    seed_bucket( );
  }    /* fresh; else state is loaded */
  int c0 = −1, period = 0, recstart = −1, recend = −1, seam_break = −1;
  int cand_p = 0, cand_c = −1;    /* pending (unconfirmed) period candidate */
  for (s = (resume_from > 0 ? resume_from : 0); s < V; s++) {
    if (recording ∧ s ≡ recstart) {
      seedn = ncur;
      seedv = xmalloc(ncur ∗ sizeof(u64));
      for (i = 0; i < ncur; i++) seedv[i] = curw[i];
```

```
      nstate[0] = ncur;
      reclev = 0;
    }
    run_step(s, recording ∧ s ≥ recstart ∧ s ≤ recend);
    if (s % m ≡ m − 1) {
      c = s/m;
      u64 h = 14695981039346656037ULL;
      long t;
      for (t = 0; t < ncur; t++) {
        unsigned char *kk = curkp + curoff[t];
        int L = curkl[t], z;
        for (z = 0; z < L; z++) {
          h ⊕= kk[z];
          h *= 1099511628211ULL;
        }
        h ⊕= #9e;
        h *= 1099511628211ULL;
      }
      colfp_g[c] = h;
      ⟨Detect and confirm the periodic column 24⟩
    }
    if (s % m ≡ m − 1 ∧ getenv("DBG"))
      fprintf(stderr, "␣␣col␣%d␣ncur=%ld␣rec=%d\n", s/m, ncur, recording);
    if (¬recording ∧ s % m ≡ m − 1 ∧ (s/m) % ckpt_every ≡ 0) save_ckpt(s + 1);
    if (recording ∧ s ≡ seam_break ∧ stop_after_record) break;
  }
  ⟨Finalize direct coverage and verify periodicity 25⟩
  direct_cov_g = stop_after_record ? direct_valid_col_g : n;
  spmv_run(Next, 1);
  return c0;
}
```

24. We commit to a period only once the boundary fingerprint has *repeated*: a lone match $colfp[c] \equiv colfp[c-p]$ can be a coincidence of the near-boundary columns, so we hold it as a candidate and require it to persist one full period further before recording. That guarantees the recorded columns sit in the bulk, far enough from the strip's right edge that the transfer is stationary; a strip too narrow to contain a confirmed period never records, which the finalizer below turns into a clear diagnostic instead of a corrupt table.

⟨ Detect and confirm the periodic column 24 ⟩ ≡

```
if (c0 < 0) {
   if (cand_p ≡ 0) {
      if (c ≥ 1 ∧ colfp_g[c] ≡ colfp_g[c − 1]) {
         cand_p = 1;
         cand_c = c;
      }
      else if (c ≥ 2 ∧ colfp_g[c] ≡ colfp_g[c − 2]) {
         cand_p = 2;
         cand_c = c;
      }
   }
   else if (colfp_g[c] ≡ colfp_g[c − cand_p]) {
      if (c − cand_c ≥ cand_p ∧ c + cand_p + 3 ≤ n) {
           /* confirmed, with bulk room for recording+seam */
         period = cand_p;
         c0 = c;
         recstart = (c0 + 1) * m;
         recend = (c0 + 1 + period) * m − 1;
         Plevs = period * m;
         recording = 1;
         period_g = period;
         c0_g = c0;
         recend_col_g = c0 + period;
         seam_break = recend + 3 * m;     /* sweep a few extra columns (direct seam) */
         fprintf(stderr, "stable␣col␣%d,␣period␣%d␣(confirmed)\n", c0, period);
      }
   }
   else {
      cand_p = 0;     /* candidate broke: restart search here */
      if (c ≥ 1 ∧ colfp_g[c] ≡ colfp_g[c − 1]) {
         cand_p = 1;
         cand_c = c;
      }
      else if (c ≥ 2 ∧ colfp_g[c] ≡ colfp_g[c − 2]) {
         cand_p = 2;
         cand_c = c;
      }
   }
}
```

This code is used in section 23.

25. The direct sweep counts every column it reaches, exact once that column's completions have all closed (a lag of a couple knight-moves). The SpMV cannot reconstruct completions that close inside the recorded/seed columns, so its *first* extrapolated column, $recend + 1$, comes out short. We therefore trust the direct count one column past the recorded region — the *seam_break* sweep ran far enough for its completions to close — and hand over to the SpMV only from $recend + 2$, exactly where the cross-check begins. If a period was recorded we also assert the transfer is stationary; a mismatch means the strip was too narrow and the recording is boundary-contaminated.

⟨ Finalize direct coverage and verify periodicity 25 ⟩ ≡

```
{
  int swept = (s ≥ V) ? n : s/m;
  if (c0 < 0) {
    direct_valid_col_g = swept > 0 ? swept − 1 : 0;    /* no period: direct only */
    fprintf(stderr, "no␣confirmed␣period␣within␣strip␣W_b=%d:␣direct␣counts␣only\
        ␣""(increase␣W_b␣to␣record␣the␣periodic␣transfer)\n", n);
  }
  else {
    direct_valid_col_g = (swept ≥ recend_col_g + 3) ? recend_col_g + 1 : recend_col_g;
    if (nstate[0] ≠ nstate[Plevs]) {
      fprintf(stderr,
          "ERROR:␣recorded␣transfer␣not␣stationary␣(nstate[0]=%ld␣""nstate[Plevs]=%ld);\
          ␣strip␣W_b=%d␣too␣narrow␣--␣recording␣columns␣are␣""boundary-contaminat\
          ed.␣Rebuild␣with␣a␣larger␣W_b.\n", nstate[0], nstate[Plevs], n);
      exit(4);
    }
  }
}
```

This code is used in section 23.

26. ⟨Subroutines 4⟩ +≡ /* SpMV from the (built or loaded) tables out to column *Nto*. *crosscheck* compares against the direct *cnt* where both are valid. Uses *cnt2* scratch. */

```
int build_extract(int mm, int Wb)
{
  return run_periodic(mm, Wb, Wb);
}
void spmv_run(int Nto, int crosscheck)
{
  int i;
  memset(cnt2, 0, sizeof (cnt2));
  if (MODP) {
    long j;
    for (j = 0; j < seedn; j++) seedv[j] = red(seedv[j]);
  }
  ⟨Iterate the SpMV out to column N 27⟩;
  {
    int good = 1, cc;
    if (crosscheck)
      for (cc = c0_g + 3; cc ≤ Nto ∧ cc ≤ direct_cov_g; cc++)
        if (cnt[cc * m] ≠ cnt2[cc * m]) {
          good = 0;
          fprintf(stderr, "MISMATCH␣c=%d␣direct=%llu␣spmv=%llu\n", cc, cnt[cc * m], cnt2[cc * m]);
        }
    for (cc = 1; cc ≤ Nto; cc++) {
      u64 val = cc ≤ direct_valid_col_g ? red(cnt[cc * m]) : cnt2[cc * m];
      if (val) printf("open␣%dx%d␣=␣%llu%s\n", m, cc, val, cc > direct_valid_col_g ? "␣␣(SpMV)" : "");
    }
    if (crosscheck) fprintf(stderr, "%s\n", good ? "SpMV␣matches␣direct" : "SpMV␣MISMATCH");
  }
}
```

27. ⟨ Iterate the SpMV out to column N 27 ⟩ ≡

```
if (c0_g ≥ 0 ∧ Plevs > 0) { u64 *v = xmalloc(nstate[0] * sizeof(u64));
memcpy(v, seedv, nstate[0] * sizeof(u64));
int basecol = c0_g + 1; while (basecol ≤ Nto) { int L; for (L = 0; L < Plevs; L++) { int
    abscol = basecol + L/m, substep = L % m;
long e;
for (e = 0; e < ncomp[L]; e++) {
  int idx = abscol * m + substep + 1 + comps[L][e].delta;
  cnt2[idx] = red(cnt2[idx] + v[comps[L][e].src] * comps[L][e].mult);
}
u64 *vn = xcalloc(nstate[L + 1], sizeof(u64)); { long ne = nedge[L];
int NT = omp_get_max_threads(), tt;
static long spl[NTMAX + 1];
spl[0] = 0;
spl[NT] = ne;
for (tt = 1; tt < NT; tt++) {
  long g = (long)((double) tt * ne/NT);
  while (g > 0 ∧ g < ne ∧ edges[L][g].dst ≡ edges[L][g − 1].dst) g++;
  spl[tt] = g;
}
#pragma omp parallel for schedule (dynamic, 1)
for (tt = 0; tt < NT; tt++) {
  long e2;
  for (e2 = spl[tt]; e2 < spl[tt + 1]; e2++) {
    int d = edges[L][e2].dst;
    vn[d] = red(vn[d] + v[edges[L][e2].src] * edges[L][e2].c);
  }
}
} free(v);
v = vn; } basecol += period_g; } free(v); }
```

This code is used in section 26.

28. ⟨ Report and cross-check 28 ⟩ ≡

```
{
  int good = 1, cc;
  for (cc = c0 + 3; cc ≤ Next ∧ cc ≤ Wb; cc++)
    if (cnt[cc * m] ≠ cnt2[cc * m]) {
      good = 0;
      fprintf(stderr, "MISMATCH␣c=%d␣direct=%llu␣spmv=%llu\n", cc, cnt[cc * m], cnt2[cc * m]);
    }
  for (cc = 1; cc ≤ Next; cc++) {
    u64 val = cc ≤ Wb ? cnt[cc * m] : cnt2[cc * m];
    if (val) printf("open␣%dx%d␣=␣%llu%s\n", m, cc, val, cc > Wb ? "␣␣(SpMV)" : "");
  }
  fprintf(stderr, "%s\n", good ? "SpMV␣matches␣direct" : "SpMV␣MISMATCH");
}
```

29. Main. No arguments: self-checks (direct sweep vs known values, and SpMV vs direct). *dualham m Wb N*: build to column W_b, then extend to column N by SpMV.

⟨ Main 29 ⟩ ≡

```
void spmv_run(int Nto, int crosscheck);
int build_extract(int mm, int Wb);
int main(int argc, char *argv[])
{
  start_mem_watcher();
  if (argc ≥ 5 ∧ ¬strcmp(argv[1], "build")) {
      /* build m Wb file [ckpt] : build+extract, dump tables */
    stop_after_record = 1;
    if (argc ≥ 6) ckpt_path = argv[5];
    if (argc ≥ 7) ckpt_every = atoi(argv[6]);
    build_extract(atoi(argv[2]), atoi(argv[3]));
    dump_tables(argv[4]);
    return 0;
  }
  if (argc ≡ 5 ∧ ¬strcmp(argv[1], "resume")) {    /* resume ckpt Wb file : continue a build */
    stop_after_record = 1;
    resume_from = load_ckpt(argv[2]);
    ckpt_path = argv[2];
    build_extract(m, atoi(argv[3]));
    dump_tables(argv[4]);
    return 0;
  }
  if (argc ≥ 4 ∧ ¬strcmp(argv[1], "run")) {
      /* run file Nto [prime] : load tables, SpMV (mod prime if given) */
    if (¬load_tables(argv[2])) {
      fprintf(stderr, "cannot␣load␣%s\n", argv[2]);
      return 1;
    }
    if (argc ≥ 5) MODP = strtoull(argv[4], 0, 10);
    spmv_run(atoi(argv[3]), 0);
    return 0;
  }
  if (argc ≡ 4) {
    run_periodic(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]));
    return 0;
  }
  struct {
    int m, Wb, c;
    u64 e;
  } chk[] = {{5, 12, 4, 82}, {5, 12, 6, 18784}, {5, 12, 8, 18061054ULL}, {5, 12, 9, 264895640ULL}, {5, 12, 10,
      7886117822ULL}, {7, 6, 4, 6378}, {6, 7, 6, 3318960}, {0, 0, 0, 0}};
  int i, bad = 0, lm = 0, lw = 0;
  for (i = 0; chk[i].m; i++) {
    if (chk[i].m ≠ lm ∨ chk[i].Wb ≠ lw) {
      run_periodic(chk[i].m, chk[i].Wb, chk[i].Wb);
      lm = chk[i].m;
      lw = chk[i].Wb;
    }
```

```
      u64 g = cnt[chk[i].c * chk[i].m], e = red(chk[i].e);
      printf("open␣%dx%d␣=␣%llu␣␣exp(mod␣p)␣%llu␣␣%s\n", chk[i].m, chk[i].c, g, e,
          g ≡ e ? "OK" : "FAIL");
      if (g ≠ e) bad++;
    }
    printf("%s\n", bad ? "SOME␣FAILED" : "ALL␣OK");
    return bad ? 1 : 0;
  }
```

This code is used in section 1.

DUALHAM